

ECED 3204 · MICROPROCESSORS

ECED3204 – Lab #1- An Introduction to Programming an Arduino Nano with VS Code/MPLAB in C and Assembly

C and AVR assembly using MPLAB Extensions, XC8, and Arduino CLI

Student name:

Student number:

Course workflow: Use Visual Studio Code with Microchip's MPLAB Extension Pack. Open an instructor-provided XC8 template, edit the source, build with MPLAB XC8, and deliberately run the global Upload task to transfer the generated HEX file through the Arduino Nano bootloader.

Pre-lab information

Read this handout from start to finish before installing anything. Set aside at least 4 GB of free disk space and use a USB data cable; a charge-only cable will power the Nano but will not create a COM port.

Do not combine random steps from older Atmel Studio instructions, Arduino IDE tutorials, and this handout.

Learning objectives

- Learn about the development environment you will be using in this class.
- Build a C program and upload its HEX file through the Nano bootloader.
- Build an AVR assembly program, upload it, and compare its behavior and code size with the C program.

Before you begin: the toolchain

Component	Purpose
Visual Studio Code + MPLAB Extension Pack	Opens the instructor-provided MPLAB project template, edits source code, and launches the build.
MPLAB XC8 compiler	Compiles C and GNU-style AVR assembly for the ATmega328P.
Arduino CLI + Arduino AVR Boards core	Uploads the generated .hex file over the Nano USB serial bootloader.
USB serial driver	Makes the Nano appear as a COM port in Windows. Many clones use CH340/CH341.

Part #1: Introduction to the Arduino Nano, the ATMEGA328P, and MPLAB in VS Code

Objective

- Familiarize yourself with the Arduino Nano and MPLAB in VS Code

Required materials

- Arduino Nano with ATmega328P
- USB data cable
- Windows 10 or 11 computer

Background Information

A microcontroller is a small computer capable of executing a limited set of specific instructions that you will program into the device. They are used in many applications, from automobile electronics to smart phones. The microcontroller used in this course is the ATmega328P, designed by Atmel (now owned by Microchip Technology). The following diagram is the pinout for the Arduino Nano: which pins on the Arduino Nano PCB (printed circuit board) go to which pins on the ATmega328P microcontroller. We will be using the Arduino Nano for all our labs, so be sure to keep this diagram handy for the rest of the semester as a reference.

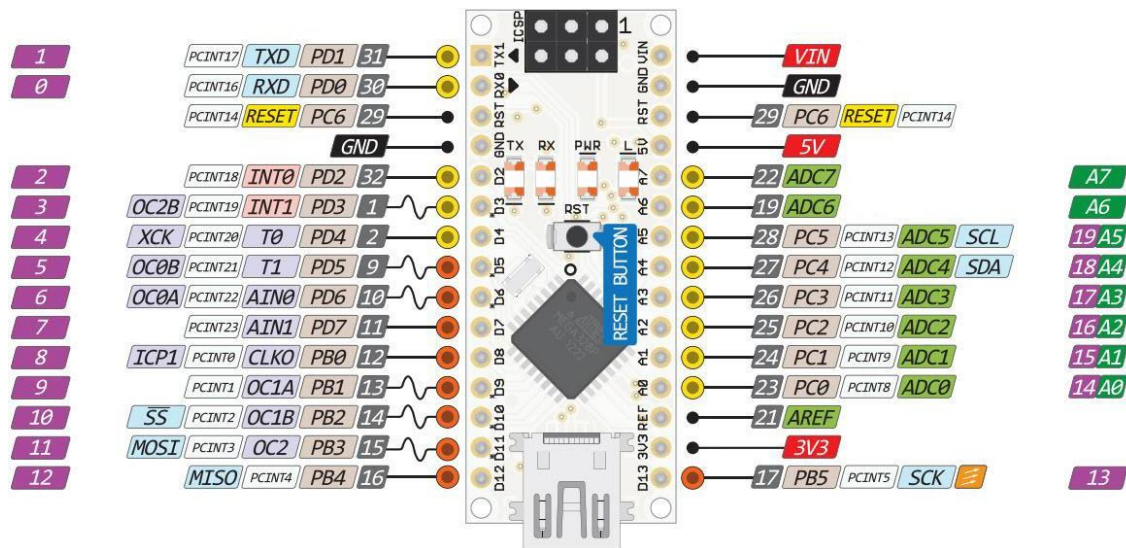


Figure 1 — Arduino Nano pinout reference. Use D8 / PB0 for this lab.

One important thing to wrap your head around is that pins on the ATmega328P do not necessarily have the same name as the pins on the Arduino Nano *board*. For example, PD2 on the ATmega328P (the square black box near the USB port) is mapped to D2 on the Arduino Nano board. The D in D2 stands for “Digital”: D2 can be a digital input or output. PB4 on the ATmega328P is mapped to D12 on the Arduino Nano, and so on. For now, the important pins on the Arduino Nano are the D pins, the GND pins (used as the ground for the Arduino Nano), and the 5V pin (this is always outputting 5V DC).

Electrical warning: Never connect an LED directly between an output pin and ground. Use the 330 Ω series resistor. The long LED lead is the anode; the short lead and the flat side of the LED body identify the cathode.

Two important reference documents are the ATmega328P datasheet:

[http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers- ATmega328P_Datasheet.pdf](http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf)

and the AVR assembly instruction set manual:

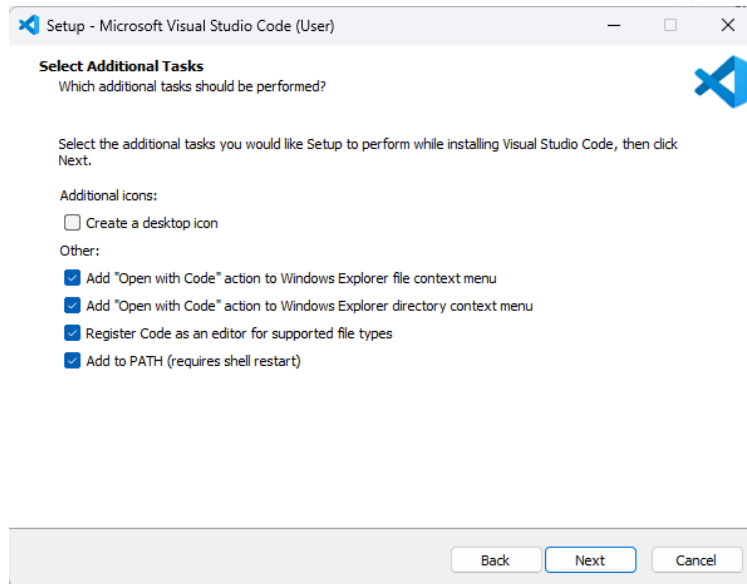
<http://ww1.microchip.com/downloads/en/devicedoc/atmel-0856-avr-instruction-set-manual.pdf>

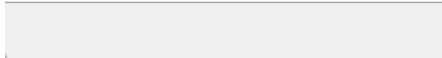
It is important that you become familiar with these documents. They are difficult to read, and the more comfortable you are with using them to find information, the easier the following labs will be.

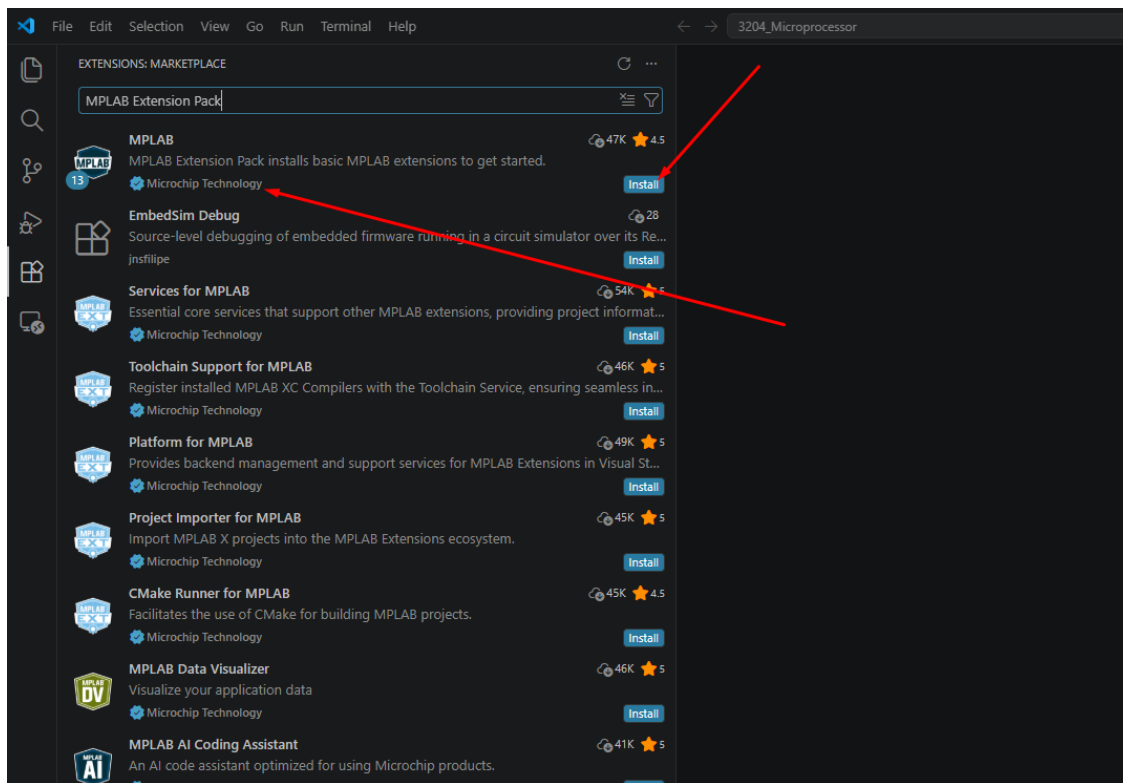
Part 2 — Install and verify the software

Step A: Install Visual Studio Code and the MPLAB Extension Pack

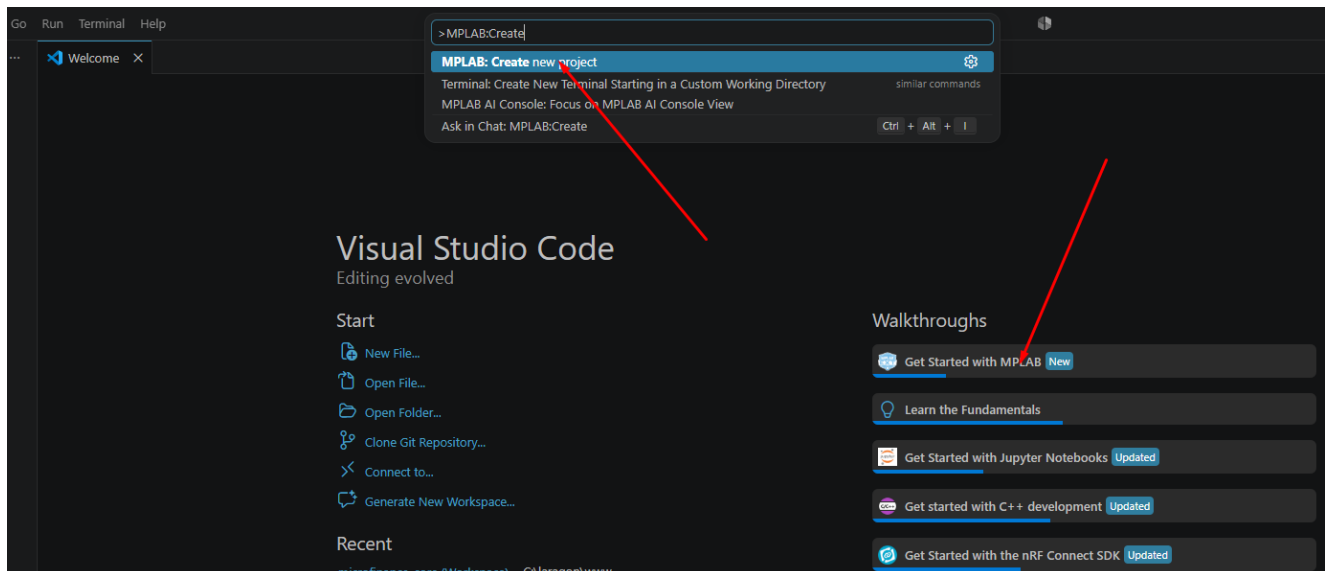
1. Download the current stable Visual Studio Code installer for Windows from Microsoft (code.visualstudio.com/download).
2. Run the installer and follow the instructions. In the Setup window, “Select Destination Folder”, it is recommended to leave the default folder. In the Setup window “Select Additional Tasks,”, add “Open with Code” to Windows Explorer as shown in the figure below.



3.  Launch VS Code. Using Ctrl+Shift+X, Open Extensions: Marketplace. Search for MPLAB Extension Pack. Select it; verify that the publisher is Microchip Technology Inc., and select Install.
4. Launch VS Code. Using Ctrl+Shift+X, Open Extensions: Marketplace. Search for MPLAB Extension Pack. Select it; verify that the publisher is Microchip Technology Inc., and select Install.



- After installing the MPLAB Extension Pack, close and reopen VS Code. Then open MPLAB either by opening the “Command Palette” (**Ctrl+Shift+P**), then typing **MPLAB:** and selecting the required command as shown in the screenshot below. If the MPLAB panel or commands appear, the extension is installed and working.



Screenshot placeholder 01 — MPLAB Extension Pack installed

Official Microchip extension guide: [MPLAB Extensions for VS Code — getting started](#)

Stop condition: Do not continue until MPLAB: Welcome appears in the Command Palette and opens successfully. If it does not, confirm the extension publisher, reload VS Code, and check that the Extension Pack and its required extensions are enabled.

Step B: Install and detect MPLAB XC8

1. XC8 is not included in the MPLAB Extension Pack. **Download MPLAB XC8 separately** (microchip.com/mplab/compilers) by selecting the appropriate version for your operating system. Run the MPLAB XC8 installation file from your “Downloads” folder. During installation, add XC8 to the system PATH if that option is available. After installation, close and reopen VS Code.

We are pleased to announce the release of MPLAB XC8 version 4.00, which includes several new features and enhancements designed to improve productivity and code quality:

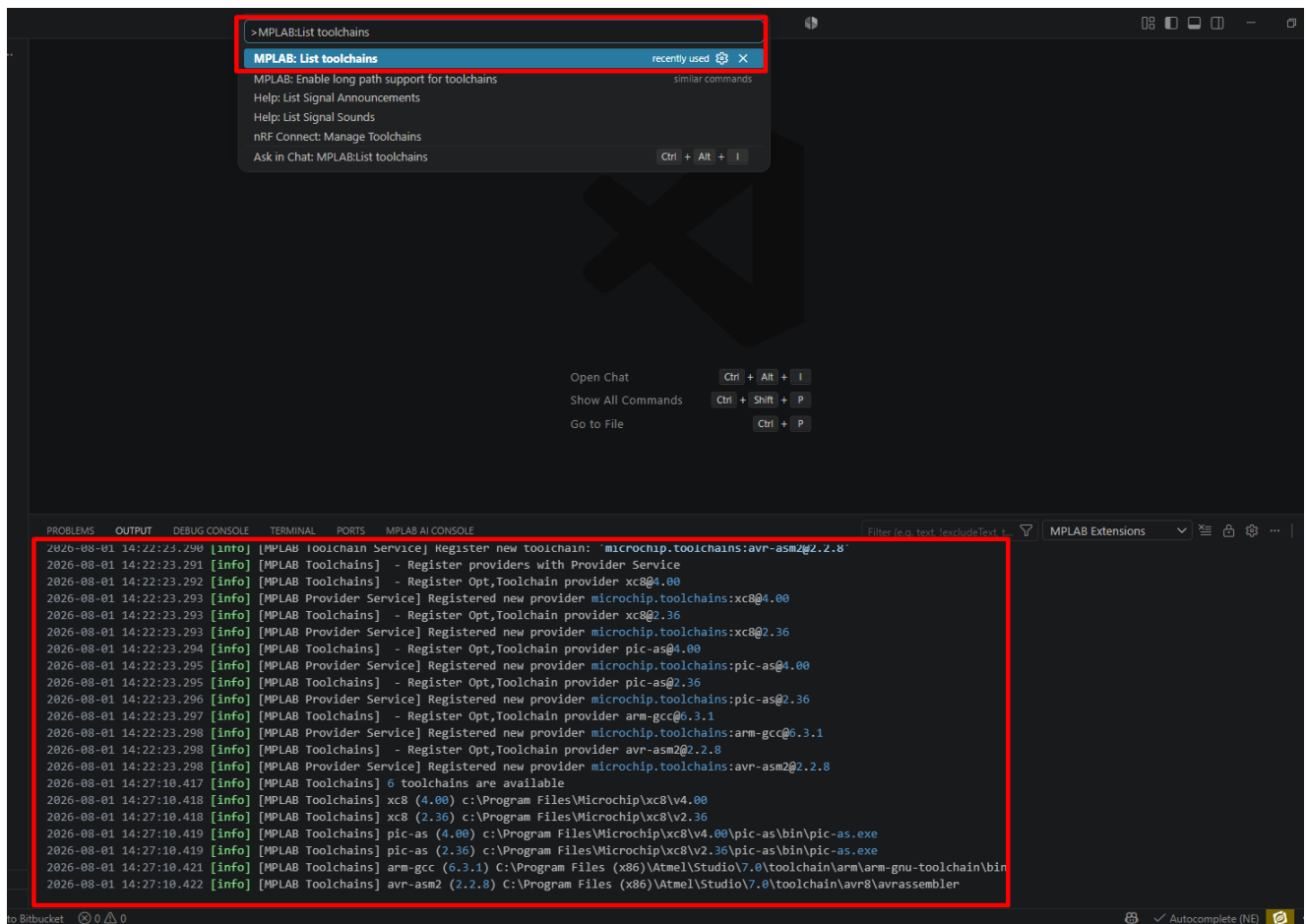
- Removed license restrictions for optimization features: All MPLAB XC compilers, including the MPLAB XC8 compiler, are now available without license restrictions, providing all users with full access to compiler optimization capabilities at no cost
- Improved assembly debugging support: Enhanced debugging support for assembly code with the ability to add DABS, DS and DB directive defined symbols to watch window, providing a better development and troubleshooting experience.
- Enhanced code optimization at higher optimization levels: Higher optimization levels produces more efficient code
- Expanded tools and diagnostics for 8-bit PIC® devices: New DQW directive for 32-bit constants in the PIC-AS assembler and a diagnostic message when branch instructions are used in assembly code.

For more information, please see the MPLAB XC8 compiler release notes.

MPLAB XC8 Compiler Downloads

Title				
MPLAB XC8 C-Compiler (Windows Installer)			2026	Download
MPLAB XC8 C-Compiler (macOS Installer)			2026	Download
MPLAB XC8 C-Compiler (Linux Installer)	735d65ba... fd06	4.00	08 Jul 2026	Download
MPLAB XC8 C-Compiler (Windows Compressed Archive)	1-110164... 500E	4.00	08 Jul 2026	Download

2. Restart VS Code after installation. Open the Command Palette (Ctrl+Shift+P), then find the MPLAB: List Toolchains. Press enter. Note that you may need to *a priori* run from the Command Palette “MPLAB: Reinitialize toolchains registration” if the compiler has not been automatically identified.
3. Confirm that the installed XC8 compiler appears in the toolchain list. If it does not, run MPLAB: Add Toolchain and select the XC8 installation folder.

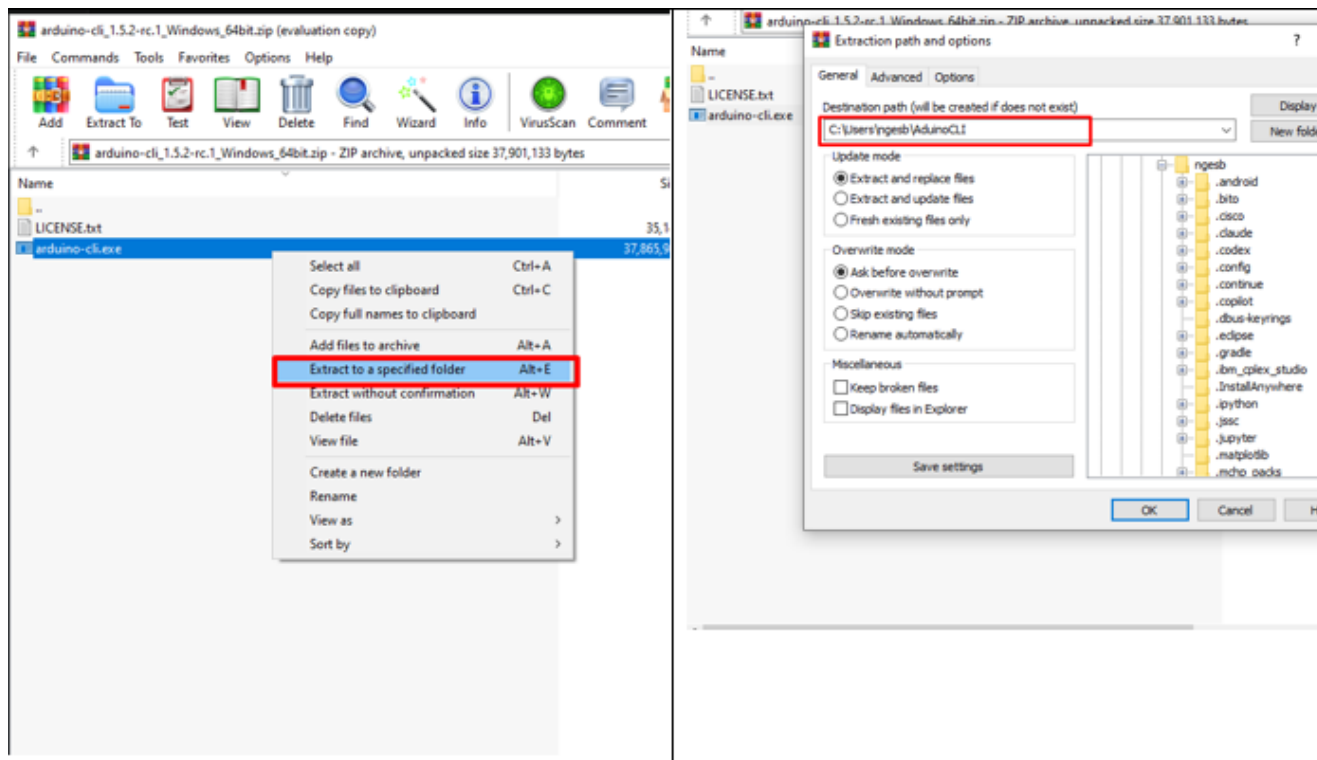


VS Code detects XC8

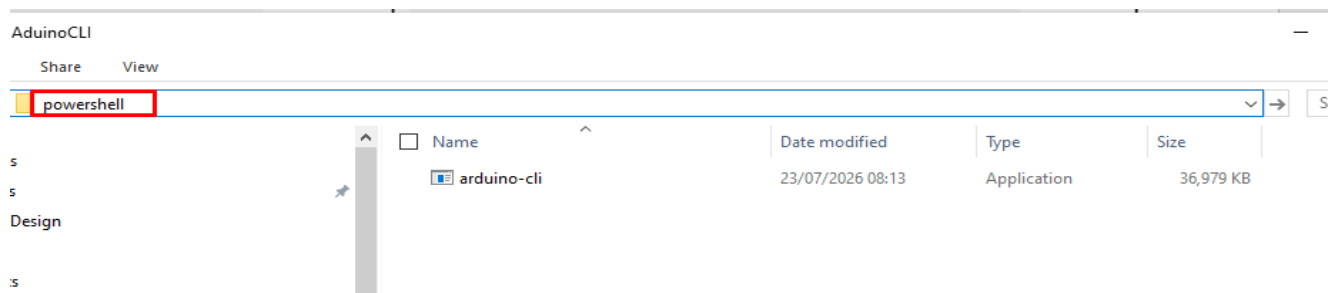
Stop condition: Do not create the project until MPLAB: List Toolchains shows XC8. Project errors caused by an undetected compiler will not be fixed by changing the C code.

Step C: Install Arduino CLI and the AVR core

1. Download the Windows 64-bit Arduino CLI ZIP from the official Arduino CLI site [Installation | Arduino Documentation](#)
2. Extract arduino-cli.exe to C:\Users\<your-name>\ArduinoCLI. Do not leave it inside the temporary Downloads ZIP window.



3. Open the ArduinoCLI folder, click the File Explorer address bar, type powershell, and press Enter.



Run the commands below one by one (press Enter between each line). Note that the first character is a period followed by a backslash. The prompt will return the status of the installation. For the third line, when prompted, accept installation of the different drivers.

```
.\arduino-cli.exe version
.\arduino-cli.exe core update-index
.\arduino-cli.exe core install arduino:avr
```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\ngesb\ArduinoCLI> .\arduino-cli.exe --version
arduino-cli Version: 1.5.2-rc.1 Commit: fef6e48df Date: 2026-07-23T11:13:25Z
PS C:\Users\ngesb\ArduinoCLI> .\arduino-cli.exe core update-index
Downloading missing tool builtin:mdns-discovery@1.0.9...
builtin:mdns-discovery@1.0.9 downloaded
Installing builtin:mdns-discovery@1.0.9...
Skipping tool configuration...
builtin:mdns-discovery@1.0.9 installed
Downloading missing tool builtin:serial-monitor@0.14.1...
builtin:serial-monitor@0.14.1 downloaded
Installing builtin:serial-monitor@0.14.1...
Skipping tool configuration...
builtin:serial-monitor@0.14.1 installed
Downloading missing tool builtin:dfu-discovery@0.1.2...
builtin:dfu-discovery@0.1.2 downloaded
Installing builtin:dfu-discovery@0.1.2...
Skipping tool configuration...
builtin:dfu-discovery@0.1.2 installed
Downloading missing tool builtin:ctags@5.8-arduino11...
builtin:ctags@5.8-arduino11 downloaded
Installing builtin:ctags@5.8-arduino11...
Skipping tool configuration...
builtin:ctags@5.8-arduino11 installed
Downloading missing tool builtin:serial-discovery@1.4.0...
builtin:serial-discovery@1.4.0 downloaded
Installing builtin:serial-discovery@1.4.0...
Skipping tool configuration...
builtin:serial-discovery@1.4.0 installed
Downloading index: package_index.tar.bz2 downloaded
PS C:\Users\ngesb\ArduinoCLI> .\arduino-cli.exe core install arduino:avr
Downloading missing tool builtin:mdns-discovery@1.1.0...
builtin:mdns-discovery@1.1.0 downloaded
Installing builtin:mdns-discovery@1.1.0...
Skipping tool configuration...
builtin:mdns-discovery@1.1.0 installed
Downloading missing tool builtin:serial-discovery@1.5.2...
builtin:serial-discovery@1.5.2 downloaded
Installing builtin:serial-discovery@1.5.2...
Skipping tool configuration...
builtin:serial-discovery@1.5.2 installed
Downloading missing tool builtin:serial-monitor@0.15.0...
builtin:serial-monitor@0.15.0 downloaded
Installing builtin:serial-monitor@0.15.0...
Skipping tool configuration...
builtin:serial-monitor@0.15.0 installed
Downloading packages...
arduino:arduinoOTA@1.3.0 downloaded
arduino:avr-gcc@7.3.0-atmel3.6.1-arduino7 downloaded
arduino:avrdude@8.0.0-arduino1 downloaded
arduino:avr@1.8.8 downloaded
Installing arduino:arduinoOTA@1.3.0...
arduino:arduinoOTA@1.3.0 installed
Installing arduino:avr-gcc@7.3.0-atmel3.6.1-arduino7...
arduino:avr-gcc@7.3.0-atmel3.6.1-arduino7 installed
Installing arduino:avrdude@8.0.0-arduino1...
arduino:avrdude@8.0.0-arduino1 installed
Installing platform arduino:avr@1.8.8...
Platform arduino:avr@1.8.8 installed
PS C:\Users\ngesb\ArduinoCLI>

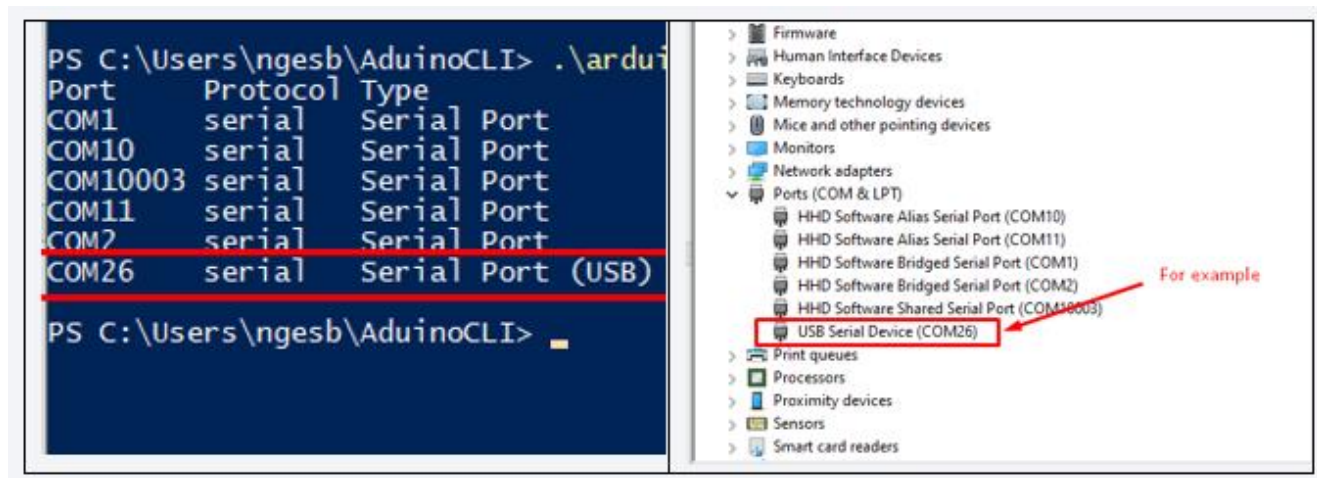
```

Arduino CLI core installation succeeds

Step D: Confirm the Nano COM port

- Connect the Nano using a USB data cable.
- Open Device Manager. Expand Ports (COM & LPT). Record the COM number shown for the Nano or its USB-serial device.
- If you are unsure which port it is, you can use the following two techniques:
 - unplug the Nano and note which port disappears; reconnect it and note which port returns.
 - In the ArduinoCLI PowerShell window, run the board detection command below.

```
.\arduino-cli.exe board list
```



Nano COM port identification

Stop condition: Do not continue until Windows shows a COM port. Reinstalling VS Code or XC8 will not fix a missing COM port. The usual causes are a charge-only cable, a missing CH340/CH341 driver, or a defective USB connector.

If the COM port is not available, this indicates that the driver is not properly installed. The **(version 3.7.2022.1)** can be downloaded from: <https://web.archive.org/web/20220519153932/https://www.wch-ic.com/downloads/file/65.html>

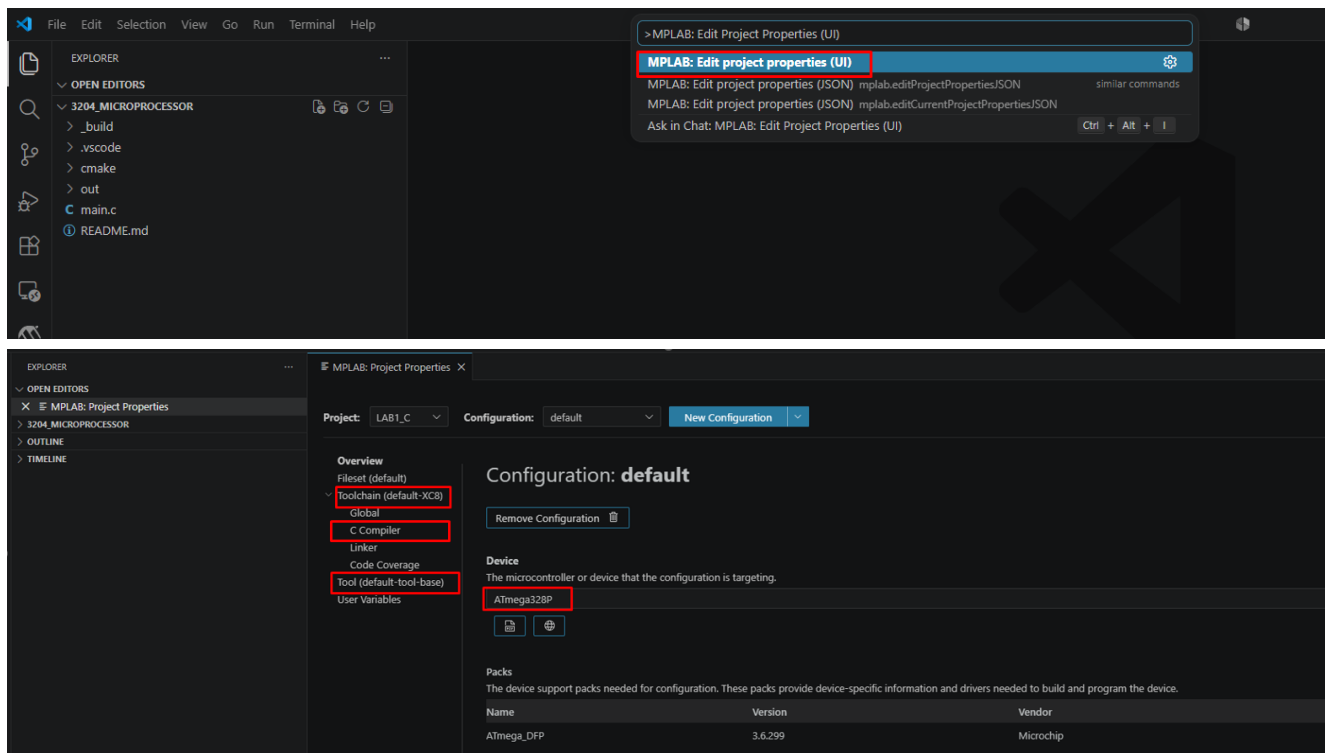
Part 3 — Open and build the instructor-provided C template in VS Code

Open the reusable C project template

1. Download Nano_XC8_C_Template.zip from the course site and extract it to a short local folder. Do not work inside the ZIP file.
2. Rename only the extracted outer folder for the current lab, for example Lab1_C. Do not rename the internal .vscode\Nano_XC8_C_Template.mplab.json file.
3. In VS Code, select File → Open Folder and open the renamed folder.

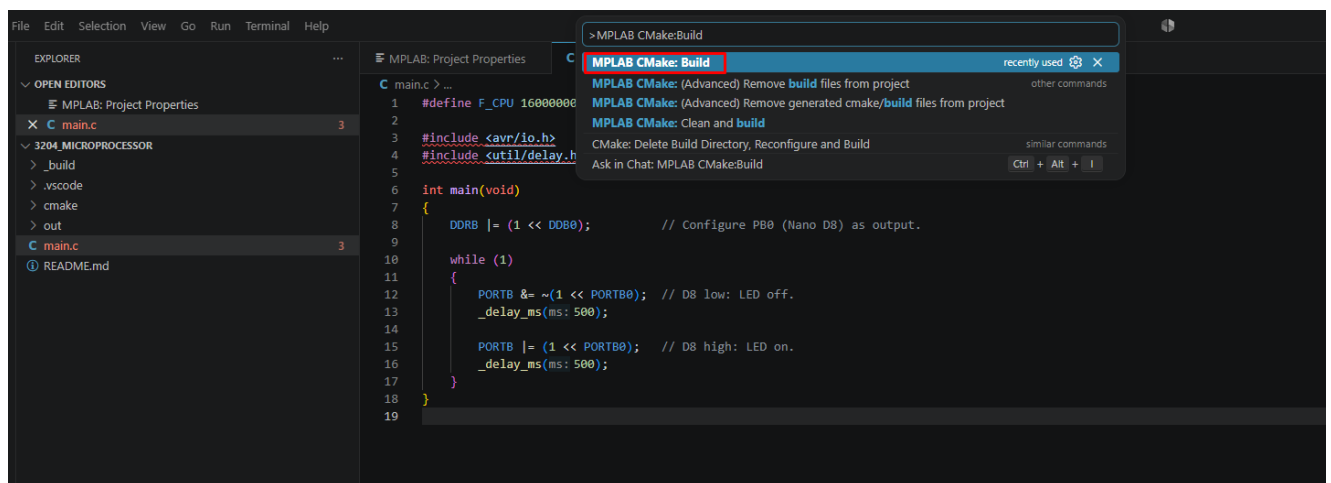
Run MPLAB: Edit Project Properties (UI) and verify the following settings:

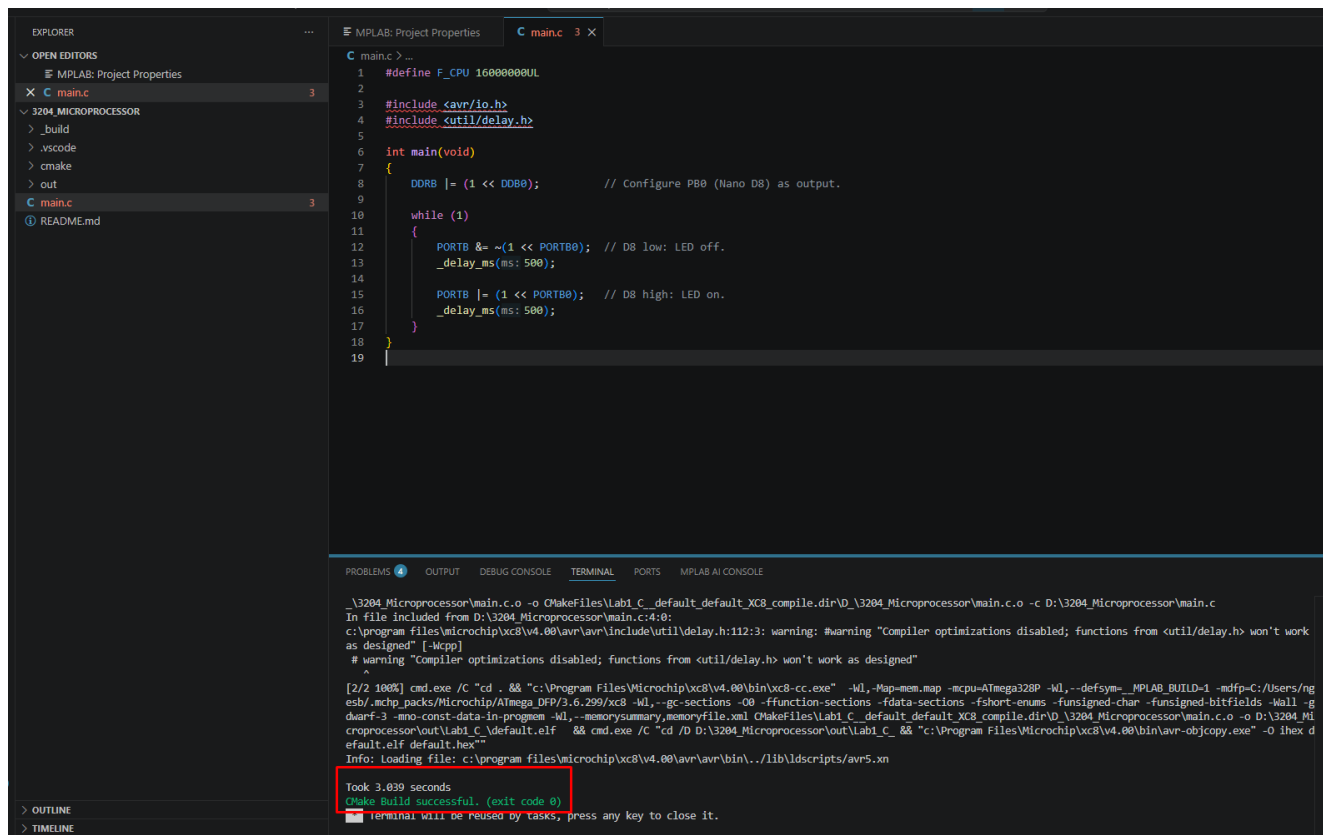
- Overview->Configuration->Device: ATmega328P,
- Tool Chain->Toolchain: XC8
- Tool Chain-> Optimization: level 1 (-O1)
- Tool -> Tool: Simulator



Screenshot placeholder 05 — Instructor-provided C template and verified MPLAB project properties

4. Open main.c and replace the placeholder program with the C program from Part 5. Save the file.
5. Press Ctrl+Shift+B (Run Build Task) and select the MPLAB-generated Build task. Continue only when the terminal reports CMake Build successful. (exit code 0).





Screenshot placeholder 06 — VS Code Build button, successful MPLAB Build, and generated HEX output

Stop condition: Do not upload until the terminal reports CMake Build successful. (exit code 0). Editor underlines are not proof of build failure or success. Read all compiler warnings.

The MPLAB extension creates the current HEX file inside the workspace's out folder. Do not manually select a HEX from another project; the global Upload task in Part 4 finds the current output automatically.

Part 4 — Configure and run the global Nano Upload task

Introduction

The project templates provided by the instructor contain the MPLAB device, compiler, optimization, and source-file configuration, but they do not contain this Upload task. The Upload task is stored globally in VS Code User Tasks because its Arduino CLI path and COM port belong to the user's computer. Configure it once during Lab 1; it will then work with every instructor template and repository opened on that computer.

A Nano bootloader profile must be chosen to upload the executable, so that they can be included in the upload script. The two options are described in the table below.

Nano type	Arduino CLI FQBN
Older official Nano or common clone	arduino:avr:nano:cpu=atmega328old (This has been set by default in the Task)
Newer official classic Nano using Optiboot	arduino:avr:nano:cpu=atmega328

Recommendation: Use the lab-kit default first: Start with `cpu=atmega328old`. If upload reports programmer is not responding or not in sync, verify the COM port and cable, then retry once with `cpu=atmega328`.

Instructions

1. Close Arduino Serial Monitor, PuTTY, or any other program using the Nano COM port.
2. Press Ctrl+Shift+P and run Tasks: Open User Tasks. Multiple options may pop up as a list. Select Others.
3. Replace the user tasks.json contents with the configuration below.

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Upload MPLAB HEX to Arduino Nano",
      "type": "shell",
```

```

"command": "$workspace = '${workspaceFolder}'; $configDir = Join-Path $workspace '.vscode';
$mplabConfig = Get-ChildItem -Path $configDir -Filter '*.mplab.json' -File -ErrorAction
SilentlyContinue | Select-Object -First 1; if (-not $mplabConfig) { throw 'UPLOAD CANCELLED: The
opened folder is not an MPLAB project root. Open the folder containing .vscode, cmake, and main.c or
main.S.' }; $outDir = Join-Path $workspace 'out'; if (-not (Test-Path $outDir)) { throw 'UPLOAD
CANCELLED: No out folder exists. Build the MPLAB project first.' }; $hex = Get-ChildItem -Path
$outDir -Filter '*.hex' -File -Recurse | Sort-Object LastWriteTime -Descending | Select-Object -First 1;
if (-not $hex) { throw 'UPLOAD CANCELLED: No HEX file was found. Build the MPLAB project first.' };
$sources = Get-ChildItem -Path $workspace -File -Recurse -Include '*.c','*.h','*.cpp','*.S','*.s' | Where-
Object { $_.FullName -notmatch '\\\\(out|_build|build|cmake)\\\\' }; $newerSources = $sources |
Where-Object { $_.LastWriteTime -gt $hex.LastWriteTime }; if ($newerSources) { Write-Host
'UPLOAD CANCELLED: Source code is newer than the generated HEX file.' -ForegroundColor Red;
Write-Host 'Run Build successfully before uploading.' -ForegroundColor Yellow; exit 2 }; Write-Host
"; Write-Host 'BUILD OUTPUT VERIFIED' -ForegroundColor Green; Write-Host \"Project:
$workspace\"; Write-Host \"HEX file: $($hex.FullName)\"; Write-Host \"Uploading to
$env:NANO_COM_PORT...\"; & $env:ARDUINO_CLI_PATH upload --port $env:NANO_COM_PORT --
fqbn $env:NANO_FQBN --input-file $hex.FullName --verify --verbose; if ($LASTEXITCODE -ne 0) {
Write-Host "; Write-Host 'UPLOAD FAILED' -ForegroundColor Red; Write-Host 'The build succeeded,
but the HEX file was not transferred.' -ForegroundColor Yellow; exit $LASTEXITCODE }; Write-Host ";
Write-Host 'UPLOAD SUCCESSFUL' -ForegroundColor Green",

"options": {
  "env": {
    "ARDUINO_CLI_PATH": "C:\\\\Users\\ngesb\\AduinoCLI\\arduino-cli.exe",
    "NANO_COM_PORT": "COM8",
    "NANO_FQBN": "arduino:avr:nano:cpu=atmega328old"
  },
  "shell": {
    "executable": "powershell.exe",
    "args": [
      "-NoProfile",
      "-ExecutionPolicy",
      "Bypass",
      "-Command"
    ]
  }
},
"problemMatcher": [],
"presentation": {
  "reveal": "always",
  "focus": true,
  "panel": "dedicated",
  "clear": true
}
}

```

```

]
}

```

4. Locate the following env section near the bottom of the file:

```

"env": {
  "ARDUINO_CLI_PATH": "C:\\Users\\<your-name>\\ArduinoCLI\\arduino-cli.exe",
  "NANO_COM_PORT": "COM8",
  "NANO_FQBN": "arduino:avr:nano:cpu=atmega328old"
}

```

Change the following computer-specific values:

- ARDUINO_CLI_PATH: Enter the complete path to arduino-cli.exe on your computer. Replace <your-name> with your Windows username. Because this is a JSON file, every Windows backslash must be written twice as \\.
- NANO_COM_PORT: Replace COM8 with the COM port assigned to your Nano. You can find it under **Device Manager → Ports (COM & LPT)** or by running arduino-cli board list.
- NANO_FQBN: Keep arduino:avr:nano:cpu=atmega328old for an older official Nano or most common Nano clones. Change it to arduino:avr:nano:cpu=atmega328 only if the board uses the newer bootloader.

5. Save the task.json file (Ctrl + s).

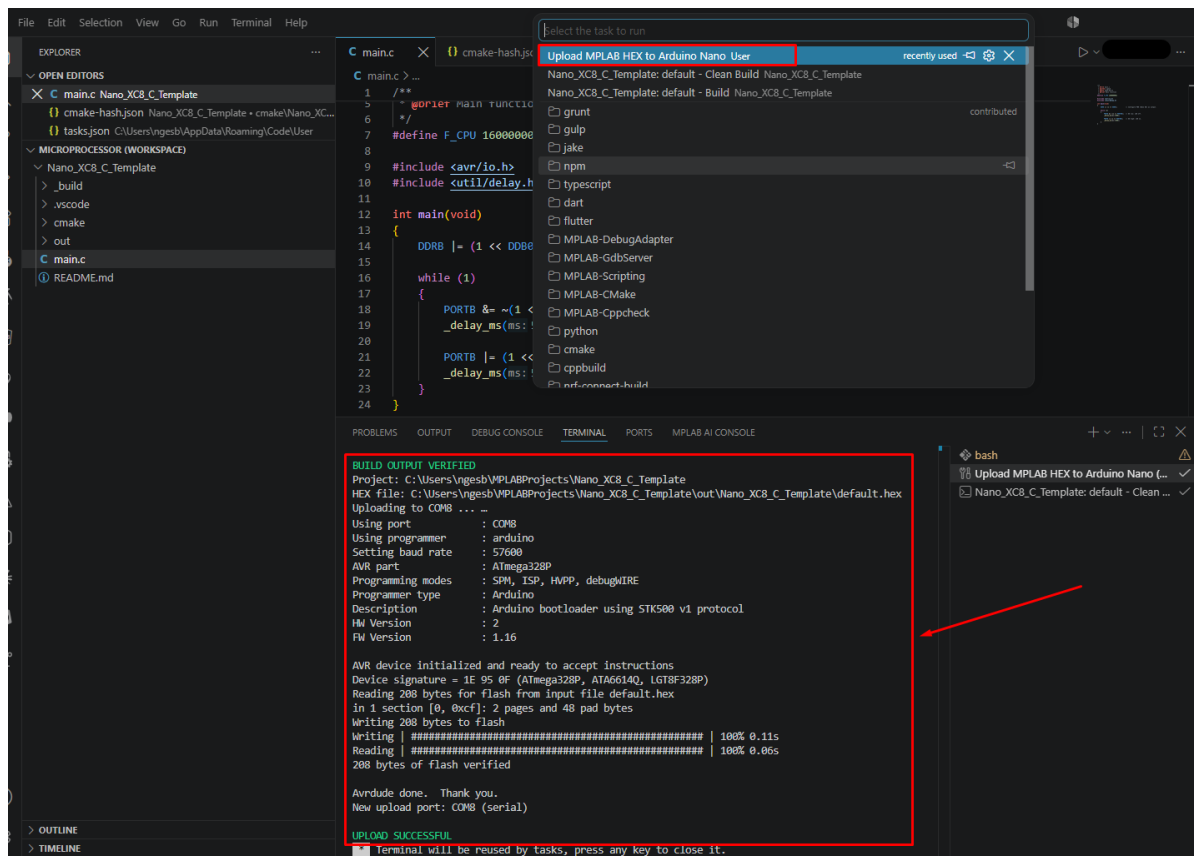
6. Open the Command Palette (Ctrl+Shift+P) and confirm “MPLAB CMake: Build” successful. (exit code 0).

7. Open the Command Palette (Ctrl+Shift+P), run Tasks: Run Task, and select Upload MPLAB HEX to Arduino Nano.

8. Confirm that the task prints BUILD OUTPUT VERIFIED, displays a HEX path inside the current workspace, and ends with UPLOAD SUCCESSFUL.

9. If the task reports that source code is newer than the HEX file, build again before uploading. If it reports UPLOAD FAILED, the code built but the transfer failed; check the COM port, USB connection, busy serial tools, and Nano profile.

Build and upload are intentionally separate actions. A successful build creates the HEX file; a successful upload transfers it to the Nano.



Screenshot placeholder 07 — Global User Task showing verified build output and successful Nano upload

Mandatory setup checkpoint

Before moving to the LED program, record the following in your lab notes:

- VS Code version
- MPLAB Extension Pack version
- XC8 version
- Nano COM port and CLI directory
- Working Nano profile: atmega328old / atmega328
- Global Upload task result and the current-workspace HEX path reported by the task

Part 5 — Programming in C: blink an LED

C source code

TYPE this code in main.c. F_CPU must be defined before including <util/delay.h>. The ATmega328P is an 8-bit MCU; the UL suffix simply makes 16000000 an unsigned long integer constant.

```

#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRB |= (1 << DDB0);           // Configure PB0 (Nano D8) as output.

    while (1)
    {
        PORTB &= ~(1 << PORTB0); // D8 low: LED off.
        _delay_ms(500);

        PORTB |= (1 << PORTB0);   // D8 high: LED on.
        _delay_ms(500);
    }
}

```

Listing 1 — Bare-metal C blink program for Nano D8 / ATmega328P PB0.

What the critical lines do

Statement	Purpose
#define F_CPU 16000000UL	Tells the delay library that the Nano CPU clock is 16 MHz.
DDRB = (1 << DDB0);	Sets only the PB0 direction bit, making Nano D8 an output.
PORTB &= ~(1 << PORTB0);	Clears only PB0 and drives D8 low.
PORTB = (1 << PORTB0);	Sets only PB0 and drives D8 high.
_delay_ms(500);	Creates an approximately 500 ms blocking delay using F_CPU.

Build, upload, and verify

1. Save main.c and run Build.
2. Fix every compiler error before uploading, and read all warnings.
3. After CMake Build successful. (exit code 0), run Upload MPLAB HEX to Arduino Nano from Tasks: Run Task.
4. Confirm that the task reports a HEX path inside the current C workspace and ends with UPLOAD SUCCESSFUL.

Build the LED circuit

1. Disconnect USB power before changing the breadboard wiring.
2. Connect Nano D8 (PB0) to the LED anode, which is normally the longer lead.
3. Connect the LED cathode to one end of the 330 Ω resistor. Connect the other resistor end to Nano GND. The LED and resistor may be swapped in series without changing operation.
4. Reconnect USB. The LED should be on for about 500 ms and off for about 500 ms.

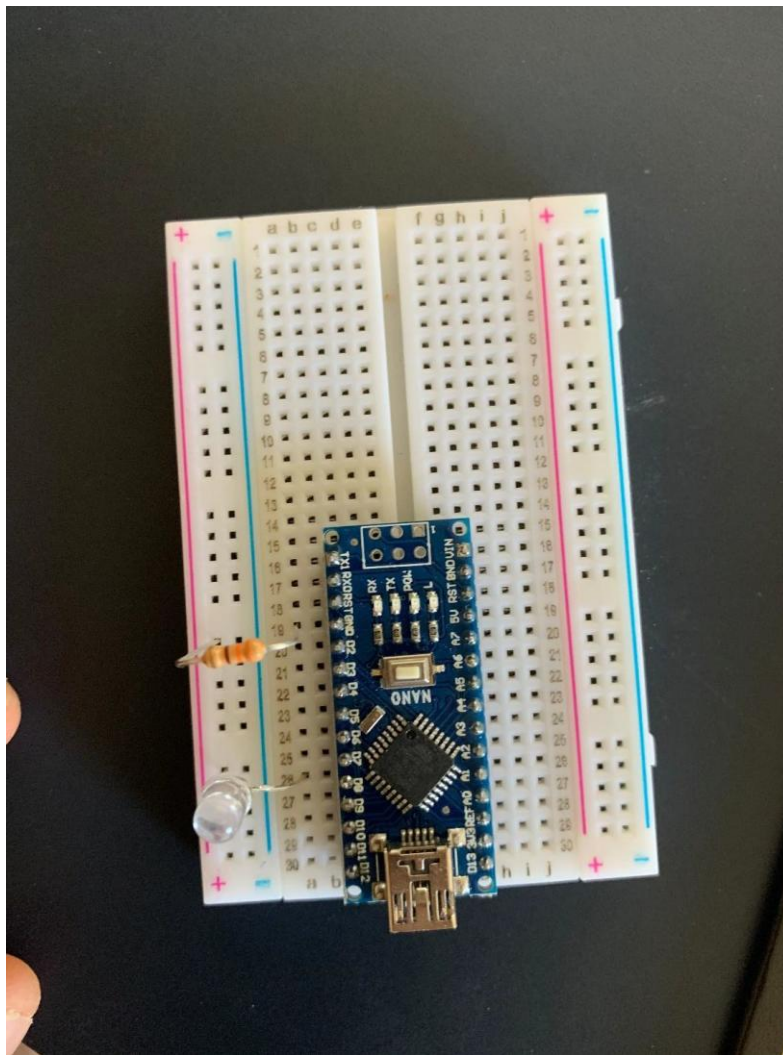


Figure 2 — Example Nano LED circuit. Verify your own D8 and GND connections against the pinout.

Part 6 — Programming in AVR assembly

Open the instructor-provided assembly template

1. Extract Nano_XC8_ASM_Template.zip to a new folder and rename only the outer folder Lab1_ASM. Do not overwrite the C project or rename the internal MPLAB JSON file.
2. Open the extracted Lab1_ASM folder in a separate VS Code window. The template contains main.S and does not contain main.c.
3. Keep the uppercase *.S extension because this source uses the C preprocessor for <avr/io.h>.
4. Replace the placeholder contents of main.S with the assembly program below (Please type it in).

```
#include <avr/io.h>

.global main
.type main, @function

main:
    sbi _SFR_IO_ADDR(DDRB), DDB0

blink_loop:
    cbi _SFR_IO_ADDR(PORTB), PORTB0
    rcall delay_loop

    sbi _SFR_IO_ADDR(PORTB), PORTB0
    rcall delay_loop
    rjmp blink_loop

delay_loop:
    // 500000 decimal = 0x07A120 (loop count, not milliseconds)
    ldi r18, 0x20
    ldi r19, 0xA1
    ldi r20, 0x07

delay_count:
    subi r18, 1
    sbci r19, 0
    sbci r20, 0
    brne delay_count
    ret

.size main, .-main
```

Listing 2 — GNU-style AVR assembly blink program for MPLAB XC8.

Expected timing difference: The assembly constant 500000 is a loop count, not a 500 ms request. At 16 MHz this simple delay is substantially shorter than the C `_delay_ms(500)` delay. That difference is intentional and is part of the lab analysis.

Build, upload, and compare

1. Save main.S and run Build. Confirm CMake Build successful. (exit code 0).
2. Run the same global Upload MPLAB HEX to Arduino Nano task. Confirm that it reports a HEX path inside the Lab1_ASM workspace.
3. Observe the LED blink rate. Record whether it is faster or slower than the C version.

- Record the byte counts or HEX file sizes reported for both builds. Use the same build configuration for a fair comparison.

Troubleshooting: use the symptom, not guesswork

Symptom	Likely cause	Corrective action
XC8 is not listed	Compiler missing or not detected	Run MPLAB: Reinitialize toolchains registration, then MPLAB: Add Toolchain if needed.
No COM port appears	Charge-only cable, driver, or USB hardware	Try a verified data cable and another port; install the correct USB-serial driver.
avrdude: programmer is not responding / not in sync	Wrong COM port, wrong Nano bootloader profile, busy port, or cable problem	Close serial tools, recheck COM, then try the alternate atmega328old/atmega328 profile once.
Build succeeds, upload fails	Build and programming are separate operations	Troubleshoot Arduino CLI, COM port, cable, and bootloader—not the C syntax.
Upload succeeds, behavior is unchanged	Wrong workspace, stale output, or incorrect HEX selected	Open the intended lab folder, run Build, and confirm the Upload task reports a HEX path inside that workspace. The stale-source check must pass.
F_CPU warning or wrong C delay	F_CPU missing, misplaced, or incorrect	Define F_CPU as 16000000UL before including util/delay.h.
LED never lights	LED reversed, wrong pin, no ground, or wiring error	Power off; verify D8/PB0, LED polarity, 330 Ω series resistor, and Nano GND.
MPLAB project will not initialize	Wrong folder opened, incomplete extraction, or extension state	Open the extracted template root containing .vscode, cmake, main.c or main.S, and README.md. Reload VS Code once if necessary; do not recreate the project.
avrdude: cannot set com-state for COMn	Incompatible newer WCH CH340 USB-serial driver	Open Device Manager → Ports (COM & LPT) → USB-SERIAL CH340 → Properties → Driver. If the installed driver is 3.9.2024.9 or another incompatible newer release, install CH340 driver 3.7.2022.1 using the archived WCH installer below. Disconnect and reconnect the Nano, confirm the driver

Symptom	Likely cause	Corrective action
		version, close serial-monitor programs, and retry Upload.

CH340 driver download (version 3.7.2022.1):

<https://web.archive.org/web/20220519153932/https://www.wch-ic.com/downloads/file/65.html>

Lab questions

1. What is the Arduino Nano CPU clock frequency in MHz?

Response:

2. What happens to `_delay_ms()` timing if `F_CPU` is defined lower or higher than the actual clock frequency?

Response:

3. Which generated HEX file contains more program data, C or assembly? Record both reported sizes and explain one reason they differ.

Response:

4. Was the assembly blink faster or slower than the C blink? Explain why the number 500000 in the assembly source is not equivalent to `_delay_ms(500)`.

Response:

5. Which ATmega328P port bit corresponds to Arduino Nano D8, and which register controls whether that pin is an input or an output?

Response:

Official software references

- [Visual Studio Code download](#)
- [MPLAB Extension Pack — Visual Studio Marketplace](#)
- [MPLAB XC compilers](#)
- [MPLAB Extensions for VS Code — getting started](#)
- [Arduino CLI documentation](#)
- [Arduino CLI upload command](#)
- [ATmega328P product page](#)